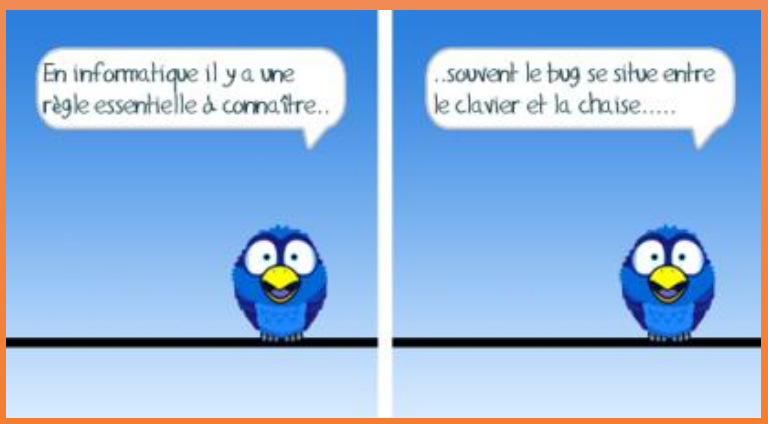
Diviser pour régner 3

Tri fusion

Vocabulaire :

- Diviser pour régner / tri fusion / terminaison et correction d'un algorithme



Prérequis :

- La notion de récursivité doit avoir été vue.
- Comprendre la notion de tri pour une liste.
- Connaître la notion de complexité temporelle d'un algorithme.

Objectifs :

- Écrire un algorithme utilisant la méthode « diviser pour régner »
- Connaître le déroulement du tri fusion et l'implémenter en exploitant la récursivité.
- Savoir que le tri fusion est un algorithme de coût en $n\log_2 n$ dans les pires des cas. Ce qui est mieux que les tris étudiés en première

Alice se décide à ranger ses bandes dessinées par ordre alphabétique de titre. Le travail est fastidieux car elle en possède une bonne centaine. Elle appelle son frère Basile à la rescousse. Ils se partagent les bandes dessinées et chacun d'eux trie sa moitié, sous la forme d'une pile de bandes dessinées. Ensuite, les bandes dessinées sont rangées dans la bibliothèque en fusionnant les deux piles, c'est-à-dire en

prenant chaque fois celle des deux bandes dessinées au sommet des deux piles qui vient avant l'autre dans l'ordre alphabétique. Si la fratrie est plus grande, on peut même imaginer décomposer le travail plus encore. Ainsi, la pile d'Alice ou de Basile pourrait être triée en étant elle-même décomposée.

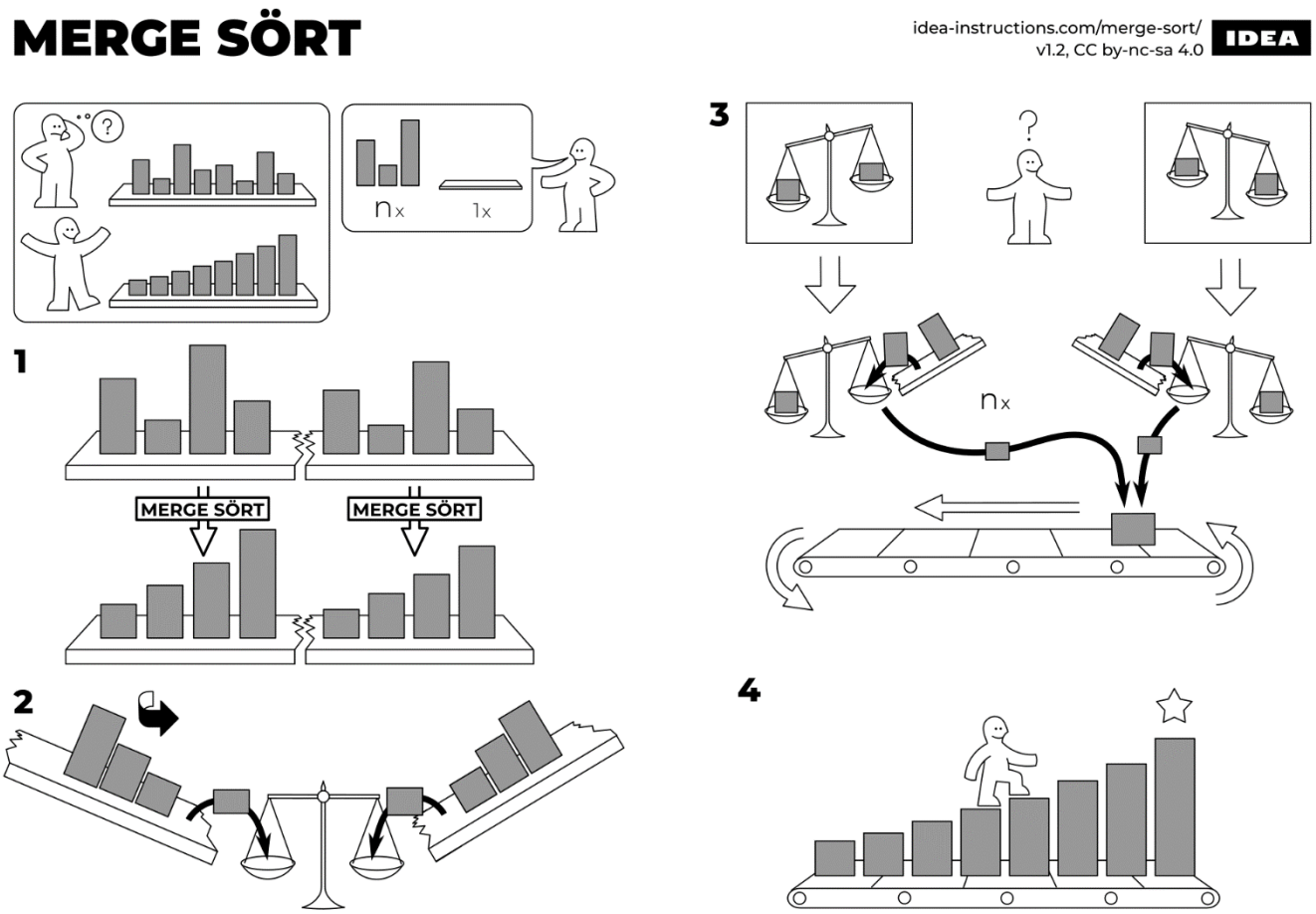
Dans un contexte informatique, cette façon de procéder suggère que plusieurs ordinateurs ou plusieurs programmes pourraient collaborer pour effectuer une tâche tel qu'un tri. C'est effectivement le cas. Mais d'une façon très surprenante, il s'avère que même un unique programme peut tirer avantage à ainsi décomposer un problème en sous-problème plus petit, ou plus simple, donc qu'il va lui-même résoudre successivement. Ainsi, même si Alice est seule pour trier ses bandes dessinées, elle peut tout à fait partager les bandes dessinées en deux tas égaux, les trier puis les fusionner. Et pour trier chacun des deux tas, elle peut recommencer ainsi avec la même idée, jusqu'à ce que chaque tas soit suffisamment petit pour pouvoir être trié sans effort. Alice vient ainsi d'inventer le tri fusion, une méthode extrêmement efficace pour trier. C'est là une instance du principe « **diviser pour régner** ». Rappelons que de manière générale, ce principe consiste à décomposer un problème à résoudre en sous-problème, plus petit, puis à les résoudre, éventuellement en appliquant le même principe autant de fois que nécessaire, puis enfin à combiner les résultats des sous-problèmes pour en déduire le résultat du problème initial.

I- LE TRI FUSION (MERGE SORT)

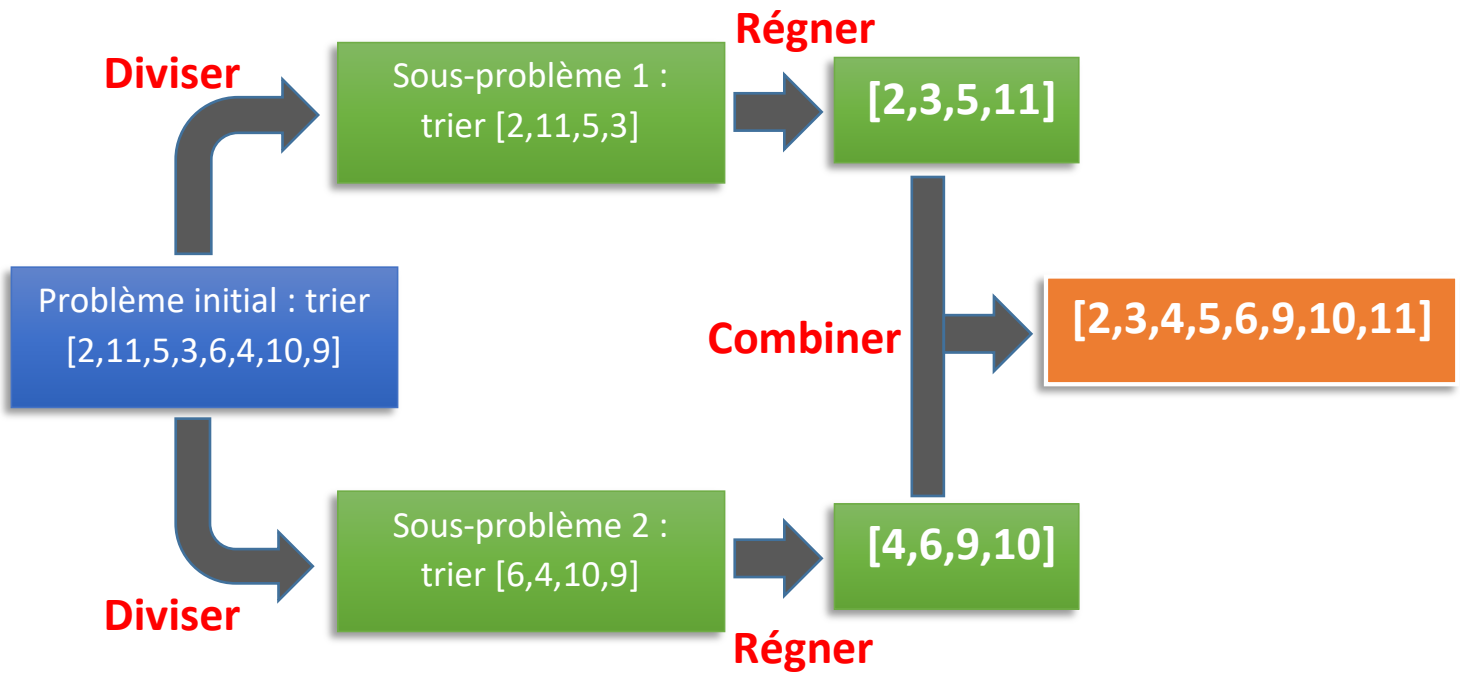
A- Principe du tri fusion

- Le **tri fusion** est un algorithme de tri. Nous appliquerons ce tri sur une liste contenant des nombres qu'il faudra classer par ordre croissant.
  Soit la liste suivante : `liste=[12,17,16,2,3,5,10,9,7,17,11,18,19]`
  Après le tri fusion, on a : `liste=[2,3,5,7,9,10,11,12,16,17,17,18,19]`
  Nous avons vu en classe de première deux algorithmes de tri : le tri par sélection (permutation en place de l'élément le plus petit de la sous-liste droite) et le tri par insertion (faire glisser vers la gauche l'élément à tester tant qu'il est plus petit dans une sous liste-gauche déjà triée). Je t'invite à faire une petite recherche pour te rafraichir la mémoire si nécessaire.
- Le tri par fusion, **merge sort** en anglais, consiste à trier récursivement les deux moitiés de la liste, puis à fusionner ces deux sous-listes triées en une seule. La condition d'arrêt à la récursivité sera l'obtention d'une liste à un seul élément, car une telle liste est évidemment déjà triée.

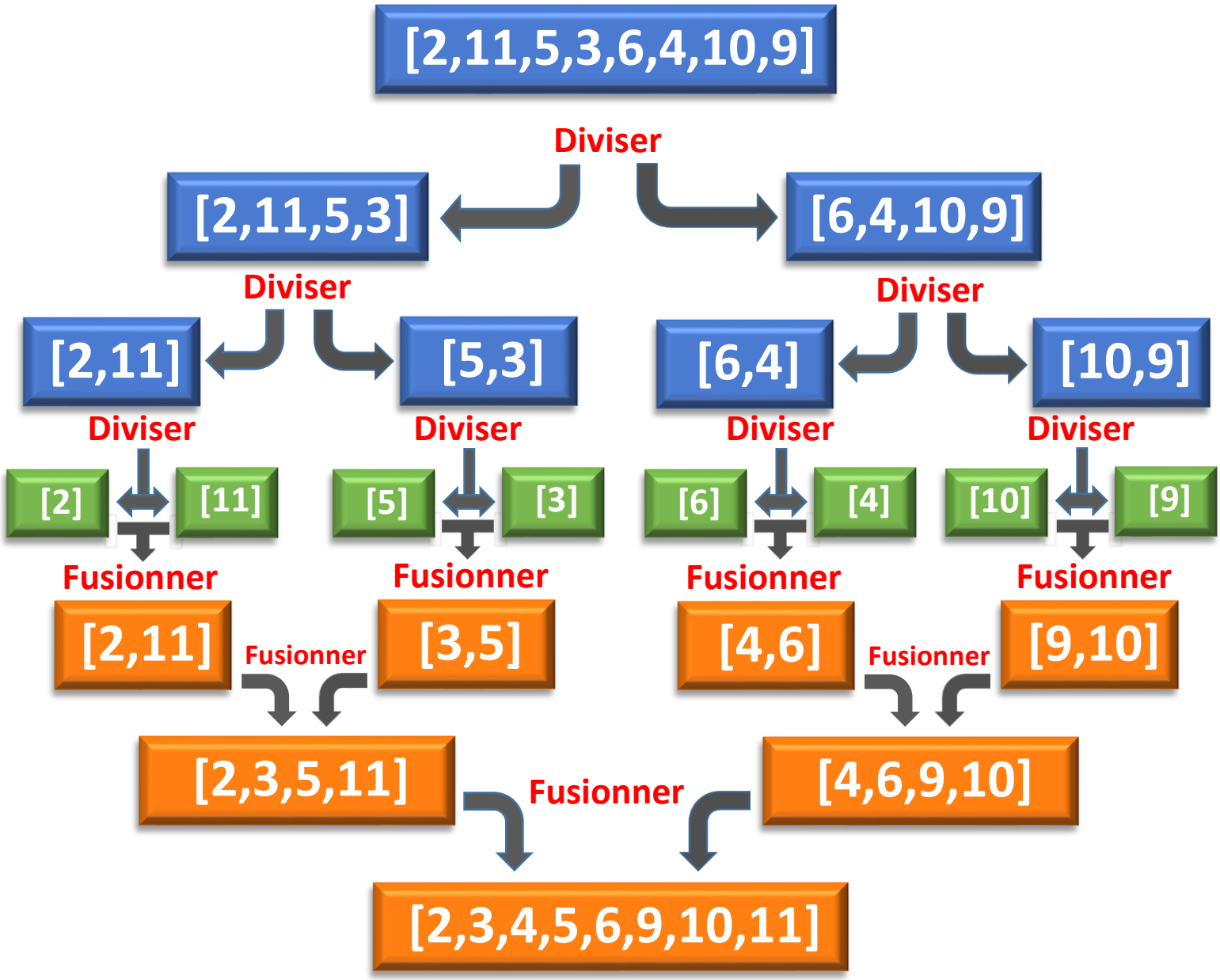
- Voici le tri fusion en image :



- Voici donc les trois étapes (diviser, régner et combiner) de cet algorithme :
  - Diviser la liste en deux sous-listes de même taille (à un élément près) en la "coupant" par la moitié.
  - Trier récursivement chacune de ces deux sous-listes. Arrêter la récursion lorsque les listes n'ont plus qu'un seul élément.
  - Fusionner les deux sous-listes triées en une seule.



- Dans le déroulement cela donne :



B- Algorithme implémenter en python

Exercice 1 :

1) Écrire sous `Pyzo`, une fonction **`fusion(liste1,liste2)`** qui avec deux listes triées en entrée, retourne une liste triée des éléments des deux listes.
2) Écrire une fonction **`diviser(liste)`** qui découpe une liste **`liste`** en deux sous-listes de longueur égales (à un élément près si la longueur est impaire).
   Aide : *le slicing peut utile pour cette partie*
3) Écrire une fonction **`tri_fusion(maListe)`** qui utilise le principe **`diviser pour régner`** et les deux fonctions précédentes.

4) Écrire une fonction `nombres_aleatoires(n,K)` qui génère une liste aléatoire d'entiers de longueur `n` avec pour valeur maximale `K`

Rappel : *Afin d'évaluer le temps d'exécution d'un algorithme, on pourra importer la fonction* `time` *du paquet* `time`*. La fonction* `time()` *ainsi importée renvoie le temps en seconde. En faisant la différence entre le temps après l'exécution et celui avant l'exécution, on peut déterminer le temps mis par l'algorithme en seconde.*

5) Écrire une fonction `duree_un_tri(l)` qui prend en paramètre une liste `l` et retourne le temps nécessaire pour que le tri par fusion soit effectué.
6) Écrire une fonction `duree_tri_fusion(n,t)` qui prend en paramètre un nombre entier `n` et un nombre entier `t`, génère `n` listes de taille `t` puis retourne le temps moyen d'un tri fusion pour ces listes. Utiliser les fonctions écrites précédemment

[Ecris directement ta réponse ici sauvegarde ensuite ton fichier pdf.]

[Ecris directement ta réponse ici sauvegarde ensuite ton fichier pdf.]

C- Complexité temporelle

- Il s'agit d'un tri très efficace pour les grandes listes.
- En pseudo-code, on peut écrire l'algorithme sous la forme suivante :

```
Algorithme : TriFusion(T)
n ← taille(T)
si n = 1 alors
    Retourner T
sinon
    T1 ← TriFusion(T[0, ⌊n/2⌋ − 1])
    T2 ← TriFusion(T[⌊n/2⌋, n])
    Retourner Fusion(T1, T2)
```

- Il y a de l'ordre de $\log_2$ n étapes de division/combinaison car comme pour le tri dichotomique, les étapes sont divisés par 2 après chaque récurrence.
  Chaque étape a un coût de l'ordre de n.
  Le **tri-fusion** a donc **une complexité en $O(n.\log_2(n))$**.
- Cela est plus efficace que les tris par sélection et le tri par insertion qui sont en $O(n^2)$.

D- Terminaison et correction de l'algorithme

- **Terminaison** de l'algorithme :
  La récursivité se fait sur des listes de taille divisée par deux à chaque récurrence, l'algorithme va donc finir par se terminer car on finit par arriver au cas de base de fin de récursivité à savoir : une liste avec un seul élément.

- **Correction** de l'algorithme :
  On voit que le cas de base donne le bon résultat puisque qu'une liste de taille 1 est déjà triée.
  D'autre part, l'algorithme se termine en arrivant sur le cas de base. Cela prouve la correction de cet algorithme.

Exercice 2 :

1) En t'inspirant de l'arbre du tri fusion précédent, dessine sur ton cahier l'arbre permettant de déterminer le maximum de cette liste sur le principe diviser pour régner.

L = [25,11,3,7,5,51,32,1,23]

2) Écris le programme en python qui permet de trouver le maximum.

[Ecris directement ta réponse ici sauvegarde ensuite ton fichier pdf.]

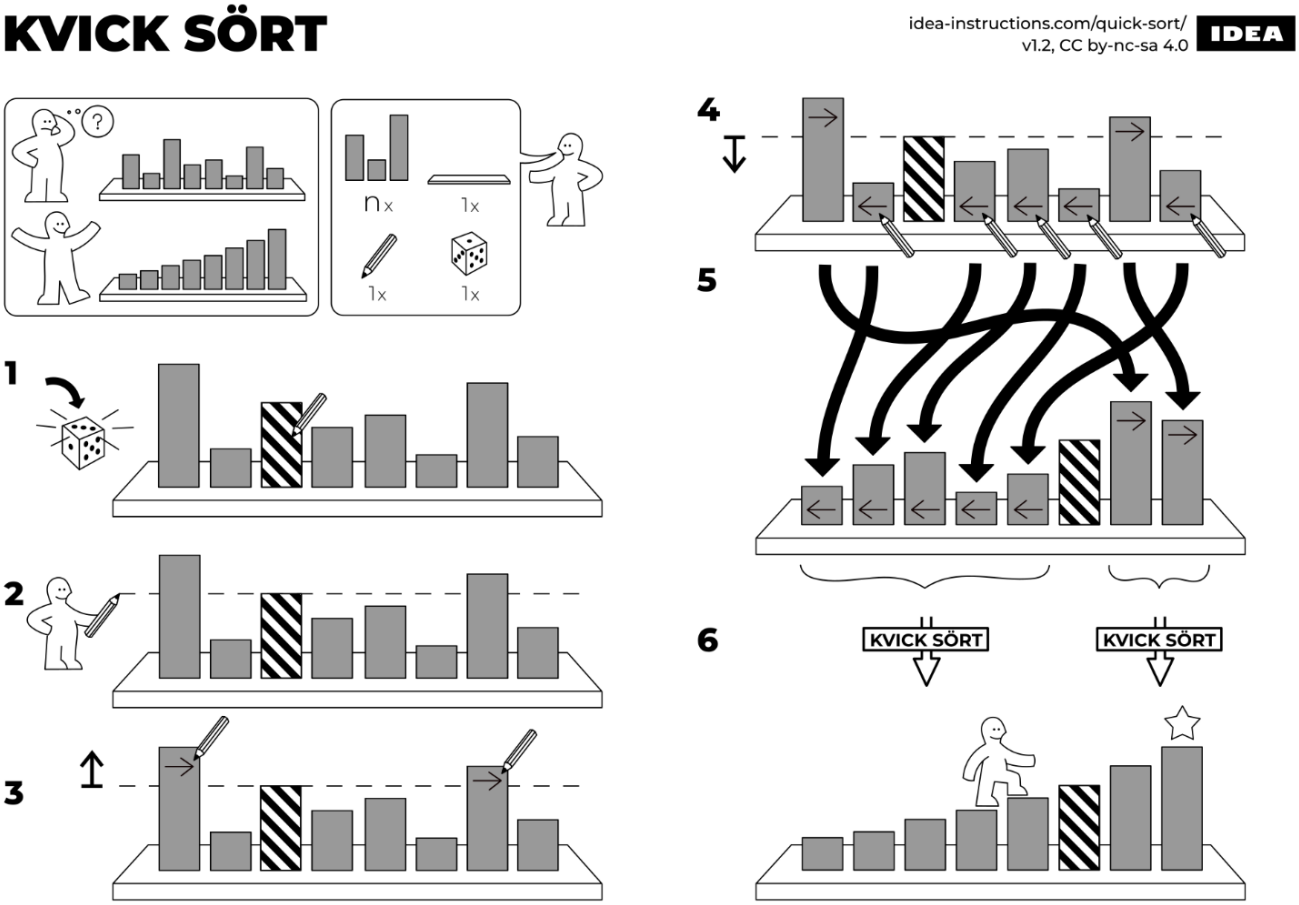
II- LE TRI RAPIDE (QUICK SORT) POUR LES PLUS RAPIDES

- Si tu as réussi à implémenter le tri fusion, voici un dernier tri qui se nomme **tri rapide**, **quick sort** en anglais. Il n'est pas au programme de Terminale mais est très utilisé pour les grandes listes.

- Voici, ci-contre, son pseudocode :

```
Algorithme : Tri_Rapide(T)
si |T|<=1 alors retourner T
créer Tab1 et Tab2
indicePivot← choix au hasard
pivot←T[indicePivot]
pour indice de T
   |si indice n'est pas indicePivot
        | si elt de T <pivot alors
mettre elt dans Tab1
        |sinon mettre elt dans
Tab2
Tab1← Tri_Rapide(Tab1)
Tab2← Tri_Rapide(Tab2)
Retourner Tab1 + pivot + Tab2
```

- Et sa représentation sous forme de dessin.



Exercice 3 :

1) Explique en une ou deux phrases le principe du tri rapide sachant qu'on le nomme parfois aussi tri pivot.
2) Écris une implémentation du tri rapide en python sous le forme d'une fonction `tri_rapide(lst)`.
3) Compare la rapidité du tri fusion à celle du tri rapide.

[Ecris directement ta réponse ici sauvegarde ensuite ton fichier pdf.]

[Ecris directement ta réponse ici sauvegarde ensuite ton fichier pdf.]

[Ecris directement ta réponse ici sauvegarde ensuite ton fichier pdf.]